TutorTrack — MVP Feature Specification

Scope and MVP boundary

TutorTrack is a CLI-first desktop application for a single private tutor managing roughly 10–40 active students. Data is stored locally in a human-editable JSON file. There is no DBMS, network service, account, or multi-user access.

The MVP supports one complete, repeatable tutoring loop: add a student, browse the roster, retrieve a student's context before a lesson, record a short outcome after it, and remove a record that is no longer needed. Five commands cover that loop: `add`, `list`, `view`, `note`, `delete`.

How the boundary was decided

The test applied is the course definition: a feature is must-have only if the product is impossible, not merely inconvenient, to use without it.

Command	MVP status	Reasoning
`add`	Must-have	Nothing exists to list, view, or annotate without it.
`list`	Must-have	The only way to see the roster and the index numbers that identify each student.
`view`	Must-have	The only way to read stored context before a lesson.
`note`	Must-have	The product's core value proposition; without it TutorTrack is an address book.
`delete`	Must-have	The only way to correct a record: with no `edit` command, a wrong phone number or a misspelt name can be fixed only by removing the

		record and entering it again. It is also the only way to discard a record the tutor should not retain.
`edit`	Deferred	Following the course's own example, delete-and-re-add substitutes for editing. The substitution is imperfect here because it discards the note history — see *Known compromises* — which is why `edit` is the first nice-to-have.
`find`	Deferred	At 40 students the roster fits in a single `list` output, so selecting by index remains workable.
`help`	Deferred	AB3's inherited help behaviour remains usable, and every error message in this specification is written to stand alone rather than instruct the tutor to type `help`.

Out of scope entirely

Scheduling, messaging, payments and invoicing, lesson materials, archiving and restoring, cloud sync, multi-user access, and management of classes, schools, or agencies.

Shared command and data rules

These rules apply to every feature. Individual features reference them rather than restating them.

Case and whitespace

Rule	Effect	Example
Command words and prefixes are case-insensitive	`ADD`, `Add`, `add` are equivalent, as are `N/` and `n/`	`ADD N/Alicia Lim …` is accepted

Stored values keep the case typed	Display shows what the tutor entered	`john doe` is stored and shown as `john doe`
Leading and trailing spaces are removed	Accidental spacing never reaches storage	`n/ Alicia Lim` stores `Alicia Lim`
Internal spaces collapse in NAME, SUBJECT, LEVEL	Spacing variants are one value	`John Doe` and `John Doe` are the same stored value
Names compare case-insensitively for duplicates	Case alone never creates a second record	`john doe` duplicates `John Doe`
NOTE preserves internal spaces	Only leading and trailing spaces are trimmed	A note keeps its own spacing exactly

Prefix parsing

- Prefixes may appear in any order. A prefix may appear at most once per command.
- A token counts as a prefix only when **both** hold: it is a recognised prefix **for the command being run**, and it appears at the start of the argument text or immediately after a space. A prefix belonging to a different command is ordinary text.
- A prefix value continues until the next token that satisfies the rule above, or the end of the command.
- Quotation marks have no special parsing meaning and are stored as ordinary characters.

The per-command prefix sets are deliberately disjoint where free text is involved:

Command	**Recognised prefixes**	**Free-text field**
`add`	`n/`, `p/`, `sub/`, `l/`, `e/`	None — all values are short and validated

`list`	None	None
`view`	None	None
`note`	`nt/` only	`nt/NOTE`, up to 500 characters
`delete`	None	None

Because `note` recognises only `nt/`, the command `note 1 nt/Revised p/e ratios and e/ notation` stores the whole sentence: `p/` and `e/` are prefixes of `add`, not of `note`, so they are ordinary characters here. In `add`, where every value is short and validated, an accidental collision surfaces as a field-validation error rather than a silent truncation.

One residual conflict remains: a note whose text literally contains `nt/` after a space is truncated at that point. Workaround: rephrase, or write the term without the trailing slash.

Roster index context

`INDEX` is a positive integer identifying a row of the **currently displayed roster**. The index context is:

- established at application launch from the full stored roster, so `view 1` works immediately without running `list` first;
- re-established by any command that redraws the roster — `list`, and a successful `add` or `delete`;
- left unchanged by any failed command, and by `view` and `note`, which do not reorder rows.

The displayed numbers and the numbers the parser accepts are therefore always the same, and no command can be blocked on having run `list` beforehand. `list` remains available as an explicit reset and becomes the filter-clearing command if filtering is added later.

The roster is always ordered by normalised student name, compared case-insensitively. Records whose names compare equal keep their creation order.

Shared messages

Situation	Message

Unknown command word	`Unknown command: <command>. Supported commands: add, list, view, note, delete.`
Unknown prefix for this command	`Unknown parameter: <prefix>. Supported parameters for <command>: <list>.`
Prefix given twice	`Parameter specified more than once: <prefix>.`
Prefix given with no value	`Value missing for parameter: <prefix>.`
Index missing	`Missing student index. Usage: <command usage>.`
Index not a positive whole number	`Index must be a positive whole number.`
Index beyond the roster	`Student index is out of range. The roster currently shows <count> student(s).`
Storage cannot be written	`Unable to save TutorTrack data. No changes were made. Check that the data file is writable and try again.`

These messages name the supported commands and parameters directly rather than referring the tutor to `help`, because `help` is a nice-to-have and may not ship.

Data integrity

Rule	Behaviour

Failed commands	Every failed command leaves the model, the GUI, and the storage file unchanged. There are no partial updates.
Storage format	A local, human-editable JSON file. No DBMS, network service, or account.
Individual malformed records at load	`Some stored records could not be read and have been skipped. Check the data file.` The valid records load; the file is not overwritten until the next successful mutation.
Whole file unreadable	`TutorTrack data file could not be read. Starting with an empty roster. The existing file will not be overwritten until you make a change.`

Shared field validation rules

Each field is validated the same way wherever it appears. Validation runs after normalisation.

Field	Acceptable values	Error message	Rationale
NAME	1–80 characters after normalisation. Must begin with a Unicode letter. Remaining characters may be Unicode letters, spaces, apostrophes, hyphens, or full stops. Internal runs of spaces collapse to one. Compared case-insensitively for duplicate detection, so `john doe` and `John`	`Name must be 1–80 characters, start with a letter, and use only letters, spaces, apostrophes, hyphens or full stops.`	Covers the forms that appear in a tutor's roster — `O'Brien`, `Nur-Aisyah`, `Tan W. K.` — while rejecting digits and slashes, which almost always indicate a mistyped command rather than a real name. The 80-character cap keeps a roster row printable in one line.

	`Doe` are the same name.		
PHONE	Required. After removing spaces and hyphens: an optional leading `+`, then 8–15 ASCII digits. `+65 9123 4567`, `9123-4567`, and `91234567` are all accepted. Stored as typed; compared for duplicates after removing spaces and hyphens.	`Parent phone must contain 8–15 digits, with an optional leading +. Spaces and hyphens are allowed.`	8–15 digits is the E.164 range, so local and overseas numbers both work without asking the tutor to pick a country. Spaces and hyphens are tolerated because tutors copy numbers from messages in varied formats, but ignored on comparison so the same number typed two ways is still one number.
EMAIL	Optional. 3–254 ASCII characters, no whitespace, exactly one `@`, a non-empty local part, and a domain containing at least one dot. Each domain label begins and ends with an alphanumeric character; hyphens are allowed inside a label. Compared case-insensitively.	`Email must be a valid address such as parent@example.com.`	Catches the common typing mistakes (missing `@`, trailing dot, stray space) without attempting to verify that a mailbox exists, which is impossible offline.
SUBJECT	1–50 characters. Unicode letters, digits, spaces, `&`, `-`, `/`, commas, and full stops. Internal spaces collapse. Stored with the tutor's case.	`Subject must be 1–50 characters and use only letters, numbers, spaces, &, -, /, commas or full stops.`	Accommodates `English Literature`, `A-Level Physics`, and `Math/Science`. `/` is permitted here without conflicting with prefix parsing, because a

			prefix is only recognised at the start of a token.
LEVEL	1–50 characters. Unicode letters, digits, spaces, hyphens, parentheses, and full stops. Internal spaces collapse. Stored with the tutor's case.	`Level must be 1–50 characters and use only letters, numbers, spaces, hyphens, parentheses or full stops.`	Curricula vary and change: `P6`, `Secondary 3`, `JC 1`, `Year 10 (IGCSE)`. A free-text field with a character allowlist is more durable than an enumerated list the tutor cannot extend.
NOTE	1–500 printable Unicode characters on a single line. Line breaks and control characters are rejected. Internal spaces are preserved exactly.	`Note must be 1–500 printable characters on one line.`	A session note is a one-line reminder, not a lesson plan. Restricting it to a single line keeps notes fast to type in a CLI, uniform to display, and scannable when several accumulate.
INDEX	A positive base-10 whole number, at least 1, matching a row of the currently displayed roster. No sign, decimal point, thousands separator, or internal space. Leading zeros are accepted (`01` is `1`).	See the shared index messages above.	An index is the fastest selector to type for a roster of this size and stays unambiguous when two students share a name. Leading zeros are accepted because rejecting them creates an error with no benefit.

Conflicts these rules create, and how a tutor works around them

Restriction	Conflict	Workaround

NAME character allowlist	A name containing a comma, a numeral, or a bracketed alias cannot be stored exactly	Enter the closest allowed spelling; nothing downstream depends on exact orthography
NOTE is one line, 500 characters	A long reflection on a lesson does not fit	Record it as two or more notes, which is also how the tutor reads it back
EMAIL requires a dotted domain	`user@localhost` and IP-literal addresses are rejected	None needed; no realistic parent contact takes that form

---

Feature 1 — Add Student

Priority: Must-have.

Purpose: Creates a student record holding the parent contact and teaching context the tutor needs to prepare for future lessons. Without it there is nothing to list, view, or annotate.

Command format

add n/NAME p/PHONE sub/SUBJECT l/LEVEL [e/EMAIL]

Example commands

add n/Alicia Lim p/+65 9123 4567 e/mrs.lim@example.com sub/Mathematics l/Secondary 3
add sub/English Literature l/JC 1 n/Dev Patel p/91234567
add n/dev patel p/98765432 sub/Physics l/JC 1

The second example shows that prefix order is free. The third is accepted even though a `Dev Patel` already exists, because the parent phone differs.

Parameters

Parameter	Acceptable values	Error message	Rationale

`n/NAME`	Required. Shared NAME rule.	Missing: `Missing required parameter: n/NAME.` Invalid: shared NAME error.	The student name is the primary human-facing identifier and appears in every roster row and confirmation message.
`p/PHONE`	Required. Shared PHONE rule.	Missing: `Missing required parameter: p/PHONE.` Invalid: shared PHONE error.	Parent contact is the one piece of information the tutor cannot reconstruct from memory, and it is half of the duplicate key. Making it required keeps every record usable for its stated purpose.
`sub/SUBJECT`	Required. Shared SUBJECT rule.	Missing: `Missing required parameter: sub/SUBJECT.` Invalid: shared SUBJECT error.	The tutor needs the subject to prepare, and it distinguishes records when the same student is taught two subjects.
`l/LEVEL`	Required. Shared LEVEL rule.	Missing: `Missing required parameter: l/LEVEL.` Invalid: shared LEVEL error.	Level carries teaching context that subject alone cannot: `Mathematics` at `P6` and at `JC 1` are different preparations.
`e/EMAIL`	Optional. Shared EMAIL rule.	Invalid: shared EMAIL error.	Phone is the required contact channel for the MVP; email is recorded when the tutor happens to have it.

Outputs — on success

- The record is created and written to storage.
- Result message: `Added student: <Name> (<Subject>, <Level>).`
- The roster redraws in case-insensitive alphabetical name order with the new student in place, and this redraw becomes the current index context, so the displayed numbers remain valid for the next command.

- The new student's profile is **not** opened automatically, so that repeated `add` commands can be typed without interruption.
- The command box is cleared and keeps keyboard focus.

Outputs — on failure

No record is created; the roster, the open profile, and the storage file are unchanged.

Failure	Message
Required prefix missing	The per-parameter message above. If several are missing, only the first in the order `n/`, `p/`, `sub/`, `l/` is reported, so the tutor fixes one thing at a time.
Value fails validation	The shared field error for that field.
Prefix given twice	`Parameter specified more than once: <prefix>.`
Prefix given with no value	`Value missing for parameter: <prefix>.`
Unrecognised prefix	`Unknown parameter: <prefix>. Supported parameters for add: n/, p/, sub/, l/, e/.`
Text before the first prefix	`Invalid command format. Usage: add n/NAME p/PHONE sub/SUBJECT l/LEVEL [e/EMAIL].`

Duplicate handling

Two students are duplicates only when **both** of the following match an existing record: the student name, compared case-insensitively with internal spaces normalised, and the parent phone, compared with spaces and hyphens removed. A duplicate is rejected with `Duplicate student: <Name> already has this parent phone.`

The rule rejects on both fields rather than on name alone because the common real cases must be accepted. Siblings share a parent phone but have different names. Unrelated students can share a common name with different parents. Name alone would block both. Requiring every field to match instead would fail to catch the actual error this rule exists to prevent — the tutor entering the same student twice and later writing session notes onto two half-complete records.

The conflict this creates: two genuinely different students who share both a name and a parent phone cannot both be stored. Workaround: distinguish them with a middle name or initial, for example `Wei Jie Tan Jr.` or `Wei Jie T. Tan`. This is an acceptable cost, since the case is rare and the workaround costs one keystroke.

Possible errors

Error	Response
Storage cannot be written (file locked, permissions, disk full)	`Unable to save TutorTrack data. No changes were made. Check that the data file is writable and try again.` The in-memory model is rolled back so it matches the file on disk.
Roster already holds the maximum supported records	Not enforced. The target user has 10–40 students; no cap is imposed and none is expected to be reached.
A value is valid but semantically wrong (a typo such as `Mathematcs`)	Not detectable. Corrected by deleting the record and adding it again — see *Known compromises* for what that costs.

Relevant UI mock-up: Mock-up A.

Feature 2 — List Student Roster

Priority: Must-have.

Purpose: Shows the full student roster and resets the numbered index context that `view`, `note`, and `delete` refer to. It is how the tutor sees who is on the roster and which number each student currently holds.

Command format

list

Example command

list

Parameters

Parameter	Acceptable values	Error message	Rationale
(none)	`list` takes no parameters, indexes, or trailing text. Trailing whitespace alone is ignored.	`Usage: list.`	Keeping `list` argument-free makes it a dependable reset point. Filtering is deferred to a nice-to-have `find`, which would set the index context the same way rather than overloading `list`.

Outputs — on success

- Every student is shown, sorted by normalised name compared case-insensitively; records whose names compare equal keep creation order, so the ordering is deterministic and repeatable for testing.
- Each row shows: index, name, subject, level, and note count.
- Result message: `Showing <count> student(s).`
- This display becomes the current index context.
- If the roster is empty, the roster panel shows `No students yet.` and the result message is `No students yet. Add one with: add n/NAME p/PHONE sub/SUBJECT l/LEVEL [e/EMAIL].`

Outputs — on failure

Failure	Message and effect
Any extra input — an index (`list 1`) or a prefix (`list n/Alicia`)	`Usage: list.`

Effect on the display	The previously displayed roster and its index context remain in place, so a mistyped `list` does not invalidate the numbers already on screen.
Effect on data	None. `list` never writes to storage.

Duplicate handling

Not applicable — `list` creates nothing. Students accepted as non-duplicates by `add` (same name, different parent phone) appear as separate rows in creation order relative to each other. Because such rows are visually identical apart from subject and level, the note count column and `view` are the means of telling them apart before running `delete`.

Possible errors

Error	Response
Individual stored records are malformed at load time	The shared storage warning is shown once at startup and the invalid records are omitted. `list` displays the valid remainder rather than failing.
A stored record has a name that no longer satisfies the current NAME rule (hand-edited JSON)	The record is treated as malformed and omitted, as above. The file is not rewritten until the next successful mutation, so the tutor can repair it by hand.
Roster too long for the visible panel	The panel scrolls. No paging or truncation is implemented, since 40 rows is the expected upper bound.

Relevant UI mock-ups: Mock-up A (populated roster) and Mock-up B (empty roster).

Feature 3 — View Student Profile

Priority: Must-have.

Purpose: Displays everything the tutor needs before a lesson for one student: parent contact, subject, level, and the session-note history. This is the read half of the product's value proposition, and the confirmation step before a `delete`.

Command format

view INDEX

Example commands

view 1
list
view 3

Parameters

Parameter	Acceptable values	Error message	Rationale
`INDEX`	Required. Shared INDEX rule: a positive whole number matching a row of the currently displayed roster.	Missing: `Missing student index. Usage: view INDEX.` Not a positive whole number (`0`, `-1`, `1.5`, `abc`): `Index must be a positive whole number.` Out of range: `Student index is out of range. The roster currently shows <count> student(s).`	An index is the fastest selector to type for a roster of this size and avoids ambiguity when two students share a name.

Outputs — on success

- The profile panel opens beside the roster; the roster stays visible and its index context is unchanged.
- Heading: `Student <INDEX>: <Name>`.
- Fields shown: parent phone; parent email, or `Not recorded` when none is stored; subject; level.
- Session notes are listed newest first. **All** notes are shown, with the panel scrolling if needed — the panel heading is `Session notes (<count>)` rather than `Recent session notes`, so it matches the behaviour. Each note shows its saved local date and time and its text.
- If the student has no notes: `No session notes recorded.`
- Result message: `Showing profile for <Name>.`

Outputs — on failure

Failure	Message
Index missing	`Missing student index. Usage: view INDEX.`
Index not a positive whole number (`0`, `-1`, `1.5`, `abc`)	`Index must be a positive whole number.`
Index beyond the roster	`Student index is out of range. The roster currently shows <count> student(s).`
Extra input after the index (`view 1 2`, `view 1 n/x`)	`Usage: view INDEX.`
Roster is empty	`Student index is out of range. The roster currently shows 0 student(s). Add one with: add n/NAME p/PHONE sub/SUBJECT l/LEVEL [e/EMAIL].`

In every case the currently open profile stays open and unchanged, and no data changes. There is no `No roster is currently displayed` failure, because the index context is established at launch and re-established by every command that redraws the roster.

Duplicate handling

Not applicable — `view` selects by position, not by name, so students accepted as non-duplicates with the same name are unambiguously distinguishable here. `view` creates nothing and merges nothing.

Possible errors

Error	Response

Index refers to a row that existed before the roster was reordered by `add` or `delete`	Cannot occur: those commands re-establish the index context as part of the same command, so screen and parser never disagree.
A stored note has a timestamp that cannot be parsed (hand-edited JSON)	The note is displayed with `Date unknown` in place of the timestamp and sorted last. It is not discarded, since the note text is the valuable part.
Stored email no longer satisfies the EMAIL rule (hand-edited JSON)	The record is treated as malformed at load time and omitted from the roster, so it cannot be viewed. The startup storage warning explains this.

Relevant UI mock-up: Mock-up C.

Feature 4 — Add Session Note

Priority: Must-have.

Purpose: Records a short outcome from a completed lesson so the tutor can resume context next time. This is the write half of the product's value proposition and the reason a general-purpose address book does not serve this user.

Command format

note INDEX nt/NOTE

Example commands

note 1 nt/Factorisation improving; revisit completing the square next lesson.
note 2 nt/Completed comprehension practice; assigned passage 4 for homework.
note 1 nt/Struggled with p/e style ratio questions and e/ notation.

The third example stores its full text. `p/` and `e/` are prefixes of `add`, not of `note`, so they are ordinary characters here.

Parameters

Parameter	Acceptable values	Error message	Rationale
`INDEX`	Required. Shared INDEX rule.	Missing: `Missing student index. Usage: note INDEX nt/NOTE.` Invalid and out-of-range messages as for `view`.	Writing a note immediately after reading the profile means the index is already on screen, so the same selector keeps the post-lesson loop to one command.
`nt/NOTE`	Required. Shared NOTE rule: 1–500 printable characters, one line, internal spaces preserved. Punctuation, digits, and `/` are all permitted.	Missing: `Missing required parameter: nt/NOTE.` Invalid: shared NOTE error.	`nt/` rather than `n/` because `n/` means NAME in `add`. A prefix that means two different things across commands is a needless memory burden for the tutor and a needless special case in the parser, and `nt/` being unique to `note` is what lets note text contain `n/`.

Outputs — on success

- A note is created with the current local date and time and appended to that student's history.
- Result message: `Added session note for <Name>.`
- If that student's profile is open, it refreshes immediately and the new note appears first.
- The roster's note count for that student increases by one. The roster order does not change, so the index context is untouched and the tutor can write a second note for the same student without re-checking the number.

Outputs — on failure

No note is created; the roster, any open profile, and the storage file are unchanged.

Failure	Message

Index missing	`Missing student index. Usage: note INDEX nt/NOTE.`
Index invalid or out of range	As for `view`.
`nt/` missing	`Missing required parameter: nt/NOTE.`
`nt/` given twice	`Parameter specified more than once: nt/.`
`nt/` with no value, or only whitespace	`Value missing for parameter: nt/.`
Note contains a line break or control character, or exceeds 500 characters	`Note must be 1–500 printable characters on one line.`
Text between the index and the prefix (`note 1 hello nt/text`)	`Invalid command format. Usage: note INDEX nt/NOTE.`

Duplicate handling

Identical note text is accepted, including on the same day and for the same student. Two notes are never treated as duplicates. Each note records a distinct lesson event and is distinguished by its timestamp and insertion order, and recurring work legitimately produces repeated wording — `Revision of past-year paper` may be true of four consecutive lessons. Rejecting matching text would suppress accurate records to prevent an error the tutor is unlikely to make and can see immediately in the profile.

Possible errors

Error	Response

Storage cannot be written	`Unable to save TutorTrack data. No changes were made. Check that the data file is writable and try again.` The note is removed from the in-memory model so it matches the file.
The tutor mistypes a note and wants to correct it	Notes are an append-only lesson log; there is no per-note edit or delete in the MVP. The tutor adds a follow-up note carrying the correction. See *Known compromises*.
Two notes are created within the same minute	Both are stored. Display order falls back to insertion order when timestamps are equal, so the ordering stays stable and testable.
The system clock is wrong or has moved backwards	The note is stored with whatever local time is reported. TutorTrack does not validate the clock, since it has no trusted reference offline, and a note with an odd timestamp is more useful than no note.

Relevant UI mock-up: Mock-up C.

Feature 5 — Delete Student

Priority: Must-have.

Purpose: Removes a student record and its session notes. It serves two needs: discarding a record the tutor should no longer retain (a student who has stopped lessons, or personal data no longer needed), and correcting a record, since with no `edit` command the only way to fix a wrong phone number or a misspelt name is to delete the record and enter it again.

Command format

delete INDEX

Example commands

list
delete 2
view 3
delete 3

The second pair shows the recommended habit: `view` the student first to confirm the right record is selected, then delete.

Parameters

Parameter	Acceptable values	Error message	Rationale
`INDEX`	Required. Shared INDEX rule: a positive whole number matching a row of the currently displayed roster. Exactly one index; ranges (`2-4`) and lists (`2 3 4`) are not accepted.	Missing: `Missing student index. Usage: delete INDEX.` Not a positive whole number: `Index must be a positive whole number.` Out of range: `Student index is out of range. The roster currently shows <count> student(s).` Extra input: `Usage: delete INDEX.`	The same selector as `view` and `note`, so the tutor learns one rule. Single-index only because deletion is irreversible: a mistyped range could remove several histories at once, and batch removal is not a need a 40-student roster generates.

Outputs — on success

- The record and all of its session notes are removed, and the change is written to storage.
- Result message: `Deleted student: <Name> (<count> session note(s) removed).` The note count is stated so that accidentally deleting a well-documented student is immediately visible rather than silent.
- The roster redraws without that student, and this redraw becomes the current index context, so the numbers on screen stay valid for the next command.
- If that student's profile was open, the profile panel closes. A profile open for a different student stays open, and its heading index updates to that student's new row number.

Outputs — on failure

No record is removed; the roster, any open profile, and the storage file are unchanged.

Failure	Message

Index missing	`Missing student index. Usage: delete INDEX.`
Index not a positive whole number (`0`, `-1`, `1.5`, `abc`)	`Index must be a positive whole number.`
Index beyond the roster	`Student index is out of range. The roster currently shows <count> student(s).`
Roster is empty	`Student index is out of range. The roster currently shows 0 student(s).`
Extra input after the index (`delete 1 2`, `delete 2-4`, `delete 1 n/x`)	`Usage: delete INDEX.`
A prefix supplied	`Unknown parameter: <prefix>. delete takes no parameters. Usage: delete INDEX.`

Duplicate handling

Not applicable — `delete` selects one existing record by position and creates nothing, so no duplicate can arise. Students accepted as non-duplicates with the same name remain separate records; deleting one leaves the other and its notes untouched. This is the reason deletion is by index rather than by name: a name could match two records, and a command that silently picked one would destroy the wrong history.

Possible errors

Error	Response

Storage cannot be written	`Unable to save TutorTrack data. No changes were made. Check that the data file is writable and try again.` The record is restored in the in-memory model so it matches the file on disk.
The tutor deletes the wrong student	Not detectable, and not reversible in the MVP — there is no undo and no confirmation prompt. The result message names the student and the number of notes removed, so the mistake is visible at once; recovery means re-adding the student, which restores the contact details but not the note history. See *Known compromises*.
The tutor wants to remove several students at once	Not supported. Repeat `delete INDEX`, checking the redrawn roster between commands, since the indexes shift after each deletion.
The deleted student's profile is open	The panel closes, so no stale profile can remain on screen for a record that no longer exists.

Relevant UI mock-up: Mock-up D.

Nice-to-have features

These are specified at outline level so the team can see what the MVP defers and in what order to pick it up if time allows.

Priority	Feature	Command	Why it is not in the MVP
N1	Edit Student	`edit INDEX [n/NAME] [p/PHONE] [e/EMAIL or e/-] [sub/SUBJECT] [l/LEVEL]`	Delete-and-re-add substitutes for it, as the course example describes. It is first here because that substitution discards the note history, which is the product's most valuable data.

N2	Find Students	`find KEYWORD [MORE_KEYWORDS]`	At 40 students the full roster is one screen. Strongest candidate for optimising the product for fast typists, since it replaces counting rows with typing a name.
N3	Undo	`undo`	Reverses the last mutation. Valuable because `delete` is irreversible, but it needs a command-history model spanning every mutating command, which is disproportionate for five commands.
N4	Command Help	`help [COMMAND]`	AB3's inherited help behaviour remains usable, and no error message in this specification depends on `help` existing.

N1 — Edit Student. Validates every supplied field before writing anything, changes only supplied values, preserves the note history, and applies the same name-plus-phone duplicate rule to the proposed post-edit values with the record being edited excluded from the comparison. `e/-` clears a stored email. On no effective change: `No changes made to student: <Name>.`

N2 — Find Students. Shows the students whose name, subject, or level contains any keyword, case-insensitively, and re-establishes the index context from the filtered result so that `view 1` refers to the first match. `list` then becomes the command that clears the filter — which is the reason `list` is kept argument-free in the MVP.

N3 — Undo. Reverses the most recent `add`, `note`, or `delete`, restoring the full record including its notes. Confirms with `Undid: <description of the reversed command>.`

N4 — Command Help. `help` lists every command with one-line usage; `help add` shows exact syntax, required and optional prefixes, and an example. Unknown command word: `No help is available for: <command>. Supported commands: add, list, view, note, delete.`

Explicitly not planned

Archiving and restoring, sample data, sibling-linking, scheduling, messaging, and payments. Archive and restore appear in the long-term user stories and would be the next substantial piece of product work after the MVP, but they introduce a second roster state that every one of the five commands would have to account for, which is out of proportion to the MVP.

Known compromises and workarounds

Every restriction below is a deliberate trade-off. Each row states the conflict it can cause for a real tutor and what that tutor does instead. This register exists so the team can answer these in a demo or peer test without improvising.

Decision	Conflict it can cause	Workaround	Why the cost is accepted
No `edit` command: corrections are made by deleting and re-adding	Fixing one digit of a phone number destroys that student's whole note history	Re-add the student and, if the history matters, copy the notes out of the JSON data file first — it is plain text and human-editable	This is the course's own MVP example applied literally. `edit` is ranked first among nice-to-haves precisely because the note history is what makes the substitution imperfect
`delete` is single-step: no confirmation prompt, no undo	A mistyped index removes the wrong student irreversibly	`view INDEX` before deleting to confirm the record; the result message names the student and note count so a mistake is visible at once	A confirmation prompt on every deletion costs the fast typist a keystroke on every legitimate use to guard against a rare mistake. `undo` (N3) is the better long-term answer
Notes are append-only: no per-note edit or delete	A typo or a wrong fact in a saved note is permanent	Add a follow-up note carrying the correction; the history reads in order	A lesson log is a record of what was observed at the time. Editing past notes makes the history unreliable, and per-note editing needs a second index scheme (student

			index plus note index) the MVP does not justify
Same name **and** same parent phone rejected as a duplicate	Two genuinely different students with the same name and the same parent contact cannot both be stored	Distinguish with a middle name or initial: `Wei Jie Tan` and `Wei Jie T. Tan`	The rule exists to catch accidental double entry, which silently splits a note history. Name-only would block siblings and common names; all-fields-match would catch nothing
NAME allows only letters, spaces, apostrophes, hyphens, and full stops, 1–80 characters	A real name containing a comma, a numeral, or a bracketed alias cannot be stored exactly	Enter the closest allowed spelling; no behaviour depends on exact orthography	An allowlist catches mistyped commands landing in the name field, which is far more frequent than an unusual name
NOTE is one line, 500 characters	A long reflection on a lesson does not fit	Split it into two or more notes	Single-line notes are fast to type in a CLI, uniform to display, and scannable when twenty accumulate under one student
A note whose text contains `nt/` after a space is truncated there	A note discussing the prefix itself is mangled	Rephrase, or write the term without the trailing slash	Inherent to prefix-based parsing. Confining `note` to a single prefix no other command uses reduces this from a frequent problem to a contrived one
Selection by index only	With 40 students the tutor scans the roster to find the right number	Keep the roster visible; it is one screen at the target size	`find` is specified as nice-to-have N2 precisely because this is the weakest point for a fast typist

Phone required, email optional	A tutor who has only an email for a parent cannot create the record	Enter a placeholder phone and correct it later	Requiring one reliable contact channel keeps every record fit for its stated purpose. Making both optional allows records that cannot do the job
Local JSON file, no sync or accounts	Records are on one machine only	Copy or back up the data file	A course constraint, and appropriate for a single-user desktop tool

---

UI mock-ups

Low-fidelity and temporary. The layout is AB3's: a persistent roster panel, a profile panel that appears after a successful `view`, a result message area, and a command box that keeps keyboard focus at launch and after every command.

Mock-up A — Roster after adding a student

```
TutorTrack — Student roster
------------------------------------------------------------------
 #  Name        Subject             Level        Notes
 1  Alicia Lim  Mathematics         Secondary 3      2
 2  Dev Patel   English Literature  JC 1             0
------------------------------------------------------------------
Result:  Added student: Dev Patel (English Literature, JC 1).
Command: add sub/English Literature l/JC 1 n/Dev Patel p/91234567
```

The roster has redrawn with Dev Patel in alphabetical position, and these numbers are the current index context. `view 2` works immediately.

Mock-up B — Empty roster

```
TutorTrack — Student roster
------------------------------------------------------------------
  No students yet.
------------------------------------------------------------------
Result:  No students yet. Add one with:
        add n/NAME p/PHONE sub/SUBJECT l/LEVEL [e/EMAIL].
Command: list
```

Mock-up C — Student profile with session notes

```
TutorTrack — Student profile
--------------------------------------------------------------------
Student 1: Alicia Lim
Parent phone: +65 9123 4567
Parent email: mrs.lim@example.com
Subject:      Mathematics
Level:        Secondary 3

Session notes (2)
18 Sep 2026, 18:35   Factorisation improving; revisit completing
                     the square next lesson.
11 Sep 2026, 18:32   Practised quadratic expansion; homework Q1-Q8.
--------------------------------------------------------------------
Result:  Added session note for Alicia Lim.
Command: note 1 nt/Factorisation improving; revisit completing the square next lesson.
```

The heading is `Session notes (<count>)`, not `Recent session notes`, because all notes are shown and the panel scrolls. A student with none shows `No session notes recorded.`

Mock-up D — Roster after deleting a student

```
TutorTrack — Student roster
--------------------------------------------------------------------
  #  Name         Subject            Level          Notes
  1  Alicia Lim   Mathematics        Secondary 3        2
--------------------------------------------------------------------
Result:  Deleted student: Dev Patel (0 session note(s) removed).
Command: delete 2
```

The profile panel has closed because the deleted student's profile was open. The roster has renumbered, and these numbers are the current index context.

Mock-up E — Error state

```
TutorTrack — Student roster
--------------------------------------------------------------------
  #  Name         Subject            Level          Notes
  1  Alicia Lim   Mathematics        Secondary 3        2
  2  Dev Patel    English Literature  JC 1               0
--------------------------------------------------------------------
Result:  Parent phone must contain 8-15 digits, with an optional
         leading +. Spaces and hyphens are allowed.
Command: add n/Sara Ng p/12345 sub/Chemistry l/Secondary 4
```

The failed command leaves the roster, the index context, and the data file untouched, and the command box retains the typed text so the tutor corrects it rather than retyping it.